Ollscoil Teicneolaíochta an Oirdheiscirt
South East Technological University

ScanIQ:

A Mobile Application for Ingredient Analysis

by

Ludmila Bulat

Student ID: 20108895

https://github.com/Ludmila20108895/ScanIQ

Final Project Academic year: 2026

Supervisor: Dave Drohan

Higher Diploma in Computer Science

Declaration of Authenticity

I hereby declare that this final project report, titled "**ScanIQ**," is my own original work, except where specific references are the work of others.

Use of Artificial Intelligence (AI) Declaration:

I wish to formally declare that I utilized Perplexity AI as an assistive tool during the research, development, and writing phases of this project. Specifically, I used the AI to help research initial technology stack options, to assist in debugging code errors (such as React state management issues), and to translate my technical notes into academic English since English is not my first language. All final software architecture, application code, UI/UX design, and core concepts described in this report remain entirely my own original work. A complete log of my interactions with the AI tool, including debugging prompts, has been included in Appendix B to ensure full transparency.

Signed:

Date: 01/04/2026

..... 1
1. Introduction 4
1.1 Project Overview and Purpose 4
1.2 Background and Motivation 5
2. Project Aim and Objectives 5
2.1 Project Aim 5
2.2 Project Objectives 6
2.3 User Interface (UI) Design Principles 6
3. Methodology 7
3.1 Introduction 7
3.2 Project Planning with Trello 7
3.3 Use of Artificial Intelligence 8
4. System Architecture 8
5. Technology Research and Selection 10
5.1 React Native vs Native Development 10
5.2 Expo Framework 10
5.3 Data Storage (Local vs Cloud) 10
5.4 External APIs 11
6. Implementation 11
6.1 Camera Integration and Permission 11
6.2 Barcode Processing and Error Handling 12
6.3 State Management (History & Favourites) 13
6.4 Dynamic Routing and UI Layout 14
7. Improvements and Enhancements 16
7.1 Product Recommendation System 16
7.2 Ingredient Analysis Engine 17
7.3 Bulk Selection and Deletion (History & Favourites) 18
7.4 Architectural Refactoring 19
8. Challenges and Problem Solving 19

8.1 API Latency and Navigation Errors 19
8.2 State Management Inconsistencies 20
8.3 JSX and UI Structure Bugs 20
9. Evaluation and Testing 20
9.1 Functional and Integration Testing 20
9.2 UI/UX and Device Testing 21
9.3 Logic Verification 21
10. Conclusion and Future Work 21
10.1 Future Enhancements 22
References: 22
- OpenFoodFacts (2026) 22
- Appinventiv (2026) 22
- Expo (2026) 22
- Foodwatch (2026) 22
- React Native Expert (2024) 22
- StackOverflow (2022) 22
- Perplexity (2026) 22
Appendix A – AI Prompt Log 23
Prompt 1 (Writing Assistance & Translation): 23
Prompt 2 (Structuring Code Explanations): 23
Prompt 3 (Formatting Architecture): 23
Prompt 4 (Researching Technologies): 23
Prompt 5 (Debugging Code): 24

1. Introduction

1.1 Project Overview and Purpose

ScanIQ is a mobile application developed using React Native and the Expo framework. The purpose of the application is to help users better understand the ingredients found in everyday food and cosmetic products.

Many consumers find it hard to understand product labels because they often contain scientific terminology and complex chemical names. As a result, people may unknowingly purchase items that are unhealthy or unsuitable for their needs.

The application allows users to scan a product barcode using their mobile device. After scanning, the app retrieves product data and generates the products score.

This process includes the following steps:

- **Barcode scanning for quick product identification**
- **A health score expressed as a percentage.**
- **Colour indicators (green, orange, red) which helps users quickly evaluate the product.**
- **Short explanations of ingredients highlighting both positive and negative aspects.**

The system is designed to be simple, accessible, and transparent and avoids commercial bias. Also does not require user authentication, making it easy to use while maintaining user privacy.

The application uses open-source data to ensure that the information is not influenced by commercial interests. The data is retrieved from:

- **OpenFoodFacts:**

Open Food Facts is an open database of food products that is created in collaboration with the users. It provides accessible information that can be reused and helps consumers make more informed decisions about what they eat. **(OpenFoodFacts, 2025)**

- **OpenBeautyFacts (planned to integrate later)**

1.2 Background and Motivation

Before choosing a specific topic for my final project, I researched many problems that I could solve through software development. One major issue I noticed is that while people increasingly want to live healthier lifestyles and know exactly what they are consuming, the market makes this very difficult. Supermarket shelves are full of attractive, beautifully packaged, and tempting products, but many of them are not as healthy as they look. I realized that everyday items contain hidden additives, high amounts of sugar, and complex chemicals that cause consumers to feel concerned and confused about their health.

The idea of creating a mobile for this project was influenced by my own personal experience. I started using an existing application called **Yuka**. I used Yuka for a good while and found it very helpful, but I noticed it had some limitations. For example, back when I was using it, the app did not provide alternative suggestions when a scanned product received a very low grade.

These gaps motivated me to create **ScanIQ** to make product information more accessible and user-friendly with the aim to:

- **Provide clearer explanations**
- **Support better decision-making**
- **Improve overall user guidance**

2. Project Aim and Objectives

2.1 Project Aim

The aim of the app **ScanIQ** was to make product information easier to understand for everyday users. Many products have long and confusing ingredient lists, and it is not always clear what is good or bad.

With this project, I wanted to create something that shows this information in a much simpler way. Instead of reading difficult labels, users can quickly see if a product is a good choice or not.

Another goal was to help people become more aware of what they buy and use every day, especially when trying to live a healthier lifestyle.

2.2 Project Objectives

To achieve this, I focused on the following objectives:

- **To allow users to scan a product barcode using their phone**
- **To get real product data from external sources like OpenFoodFacts**
- **To create a system that checks ingredients and gives a score**
- **To show results in a simple way using colours and percentages**
- **To explain ingredients in a short and clear way**
- **To let users save products (favourites and history)**
- **To keep the app easy to use and not complicated**

These objectives guided the development process and ensured that the application met its intended purpose. They are also influenced by common features found in similar applications and development guides for product scanning apps (Appinventiv, 2026).

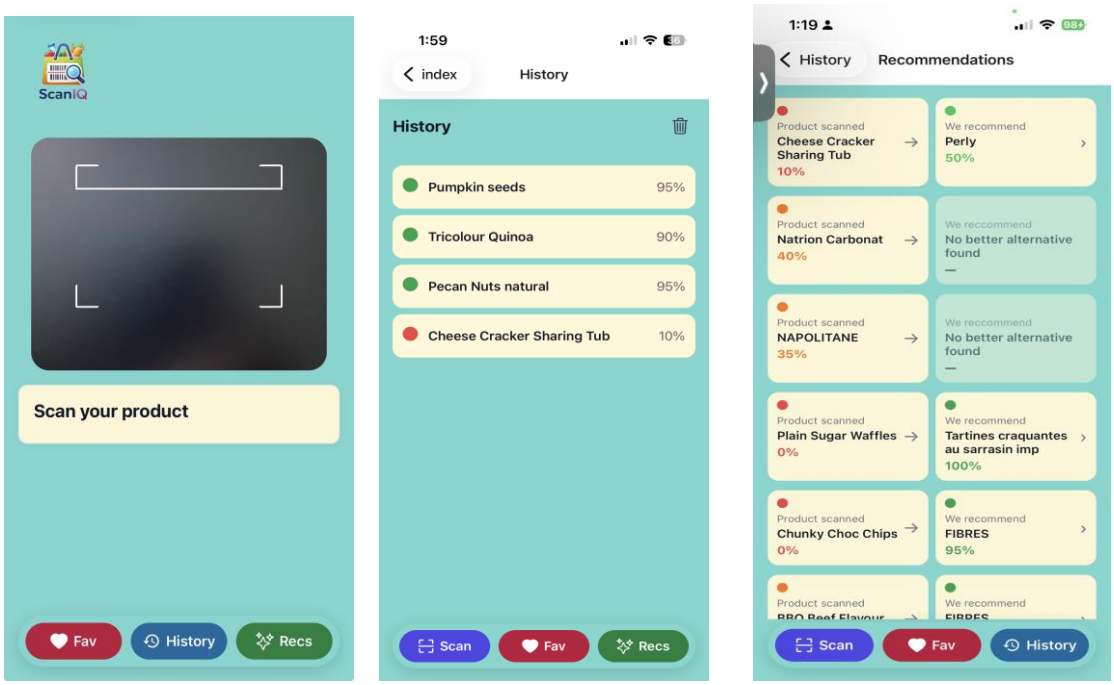
2.3 User Interface (UI) Design Principles

A major focus of this project was ensuring the user interface was accessible to people who are not technically or scientifically literate. To achieve this, I applied several core UI design principles:

Card-Based Layout: Instead of presenting users with walls of text, information is broken down into distinct, rounded UI cards. This visually separates the camera view, the product results, and the navigation buttons.

Traffic Light Colour Coding: I utilized a universal colour language. Red immediately communicates danger or high risk (e.g., high sugar), orange indicates caution, and green indicates safety or health (e.g., fibre). This removes the need for the user to understand complex nutritional values.

Bottom Navigation: To make the application feel like a native, professional iOS/Android app, I implemented a persistent bottom navigation bar. This ensures that the user is never more than one click away from their core features (Scan, History, Favourites, Recommendations).



3. Methodology

3.1 Introduction

When I started building ScanIQ, I knew I needed a good plan, not just code. I decided to follow a simple version of the **Agile methodology**. Instead of trying to build the whole app at the same time, I broke the work down into smaller, weekly tasks. This made the project feel much easier to manage because I could focus on finishing just one feature at a time. To keep everything organized, I used Trello to track my daily tasks and GitHub to save my code safely. This combination really helped me see my progress and fix mistakes without losing my work.

3.2 Project Planning with Trello

I used Trello as my main tool to organize my thoughts and plan my coding sessions. I set up a board and divided the project into clear sprints. My main steps were:

- Researching and planning the app
- Setting up the React Native environment
- Making the barcode scanner work
- Connecting to the OpenFoodFacts API
- Building the safety scoring system
- Designing the user interface

- Testing the app and fixing bugs

Every time I finished a piece of code, I moved the task card in Trello to "Done." This gave me a very clear picture of what was finished and what I still needed to do. To be honest, the coding did not always go exactly to plan. Some features, like the API integration, took me much more time than I expected, while other weeks went faster. Even when my schedule changed, having **Trello** kept me focused and flexible.

3.3 Use of Artificial Intelligence

During the development and documentation of this project, I utilized Perplexity AI as an assistive tool. Since English is not my first language, I used AI to help translate my own technical notes into natural English for this report. I also used it to help research frameworks and brainstorm solutions when debugging code issues, such as React Native state management. All final implementation and logic remain my own work. A full log of the prompts and responses used can be found in **Appendix B (Perplexity, 2026)**.

4. System Architecture

When building ScanIQ, I organised the application in a modular way so that each feature has a clear role and responsibility. This made the project easier to understand and also helped during development when adding or updating features. Instead of using a complex backend system, I decided to keep the architecture lightweight and handle most of the logic directly inside the application.

The main flow of the application starts when the user opens the app and accesses the scanning feature. At this point, the system first checks if the device camera permission has been granted. If not, the user is asked to allow access before continuing. Once permission is given, the camera view is displayed and the app is ready to scan a barcode.

The core process can be summarised as follows:

- **Check camera permission**
- **Open camera and scan barcode**
- **Send request to external API**
- **Process returned data**
- **Generate product score**
- **Display results and store data**

When a barcode is detected, the application sends a request to the OpenFoodFacts API using the product barcode. This is done through the **/api/v0/product/{barcode}.json** endpoint, which returns product information in JSON format (OpenFoodFacts, 2026). This step allows the app to retrieve real product data instead of relying on static or hardcoded values.

After receiving the data, the application processes it and maps it into a structured format using a Product model. This makes it easier to work with the data inside the app. At this stage, a scoring system is applied. The score is calculated by checking both negative and positive aspects of the product.

For example:

- **Negative factors:** sugar, salt, saturated fat
- **Positive factors:** fibre and other beneficial ingredients

Based on this calculation, the product is classified into categories such as Bad, Poor, Good, or Excellent. This helps simplify the information and makes it easier for users to quickly understand the result without needing to read complex details.

Once the analysis is completed, the product is saved locally so that it can be accessed again later. Instead of using a full database, I used a simpler approach based on in-memory state management. This was enough for the scope of the project and kept the application fast and responsive.

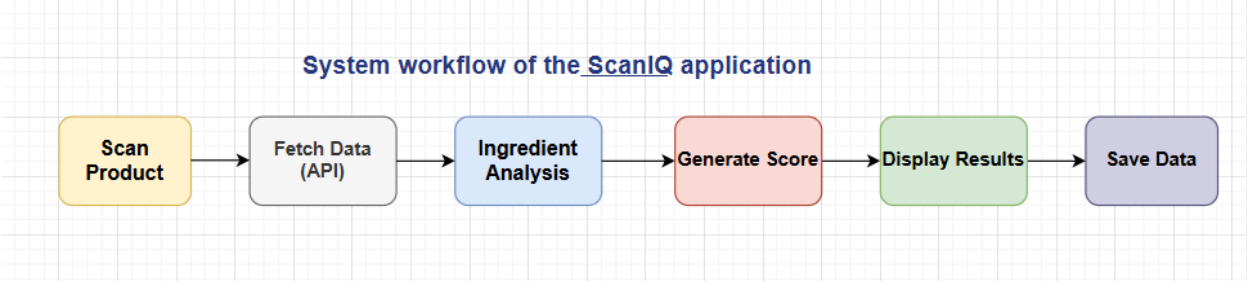
The data structure is divided into two main parts:

- **History Store**
  - Stores all scanned products
  - Includes timestamp (scannedAt)
  - Allows users to view previous scans without new API calls
- **Favourites Store**
  - Stores only product IDs
  - Links to existing product data
  - Avoids duplication and keeps storage lightweight

This approach keeps the data handling simple while still supporting the main features of the application. It also reduces unnecessary API calls and improves performance.

Overall, this modular structure allows different parts of the application, such as scanning, data processing, and storage, to work together smoothly. It also makes it easier to extend the app in the future by adding new features without changing the entire system.

Figure 1:



5. Technology Research and Selection

Before beginning development, I researched several options for building the mobile application. I needed a framework that would allow me to build for both iOS and Android efficiently.

5.1 React Native vs Native Development

Developing native applications requires writing separate code for iOS (using Swift) and Android (using Kotlin). Because I had a strict time limit, I researched cross-platform frameworks. I chose React Native because it uses JavaScript/TypeScript, languages I am familiar with. It is component-based, has a massive community, and offers fast refresh features that drastically speed up development.

5.2 Expo Framework

While React Native is powerful, configuring native build files can be difficult. I chose to use the Expo framework on top of React Native.

Pros of Expo*:* It provides pre-built libraries for device hardware (like **expo-camera**), offers file-based routing (**expo-router**), and allows testing directly on a physical phone over Wi-Fi using the Expo Go app.

Cons: It can sometimes result in a larger app size, but for ScanIQ, the benefits of rapid development heavily outweighed this limitation.

5.3 Data Storage (Local vs Cloud)

Instead of using a heavy cloud database like Firebase with user authentication, I decided to use local device storage. This decision aligned with the project's goal of protecting user

privacy and removing the need for account creation. I used a shared in-memory store and **AsyncStorage** to manage the users history and favourites locally.

5.4 External APIs

For the product data, I integrated the OpenFoodFacts API.
Pros: It is a massive, global, open-source database that provides detailed ingredient lists and nutritional data without any commercial bias or subscription fees.

6. Implementation

The development of **ScanIQ** was carried out in small and manageable stages rather than trying to build everything at once. This made it easier to focus on one feature at a time, test it properly, and then move on to the next part.

Below is a breakdown of how the main technical features were implemented during the project.

6.1 Camera Integration and Permission

The first major feature implemented was the barcode scanner. To achieve this, I used the **CameraView** component provided by the Expo framework. Since this feature requires access to the device camera, it was necessary to handle user permissions properly.

I used the useCameraPermissions hook to manage this process. When the ScanScreen loads, a **useEffect** hook checks the current permission state:

- If permission is not yet determined, the app automatically requests it
- If permission is denied, the app shows a message and a button to allow access
- If permission is granted, the camera view is displayed

This ensures that the app does not crash or behave unexpectedly if the user denies access. Instead, it provides a simple and clear interface explaining what is required. This approach follows the structure recommended in the Expo documentation (Expo, 2026).

Figure 2:

```
// main screen
export default function ScanScreen() {
  const [permission, requestPermission] = useCameraPermissions();
```

```
useEffect(() => {
  if (!permission) {
    requestPermission();
  }
}, [permission, requestPermission]);
```

```
if (!permission) {
  return (
    <View style={{ flex: 1, backgroundColor: "#72D8CF", padding: 16 }}>
      <Text style={{ color: "#888888" }}>Checking camera permissions...</Text>
    </View>
  );
}

if (!permission.granted) {
  return (
    <View style={{ flex: 1, backgroundColor: "#72D8CF", padding: 16 }}>
      <Text style={{ color: "#888888", marginBottom: 12 }}>
        Camera access is required to scan products.
      </Text>
      <Button title="Grant Camera Access" onPress={requestPermission} />
    </View>
```

6.2 Barcode Processing and Error Handling

Once the camera is active, the barcode scanning process begins using the onBarcodeScanned callback. This function is triggered whenever a barcode is detected by the camera.

Figure 3:

```
const handleBarCodeScanned = async (result: BarcodeScanningResult) => {
  if (isScanning.current) return;
  isScanning.current = true;

  const data = result.data ?? "";

  if (data.length < 5) {
    setTimeout(() => {
      isScanning.current = false;
    }, 3000);
    return;
  }
  try {
    setNotFound(false);
```

One issue I encountered during development was that the scanner could trigger multiple times very quickly, which could cause performance problems. To solve this, I implemented a control mechanism using a boolean state variable called scanned.

- **When a barcode is detected / scanned is set to true**
- **While scanned is true / further scans are ignored**
- **When the user presses "Scan again" / scanning is re-enabled**

This pattern is commonly used in React Native applications and is recommended in community solutions (StackOverflow, 2022).

In addition, I added some basic validation to improve reliability. For example:

- **If the barcode is too short / it is ignored**
- **If no valid product is found / a "Product not found" message is displayed**

This prevents unnecessary API calls and improves the overall user experience.

6.3 State Management (History & Favourites)

To manage data across different screens, I created simple and lightweight store modules. This approach was sufficient for the scope of the project and kept the application easier to manage. Two main stores were implemented:

- **Favourites Store (favouritesStore.ts)**

The **favouritesStore.ts** module contains an internal array of strings (**favouritesIds**). I created a **toggleFavourite(id)** function that checks if an ID currently exists in the array; if it

does, it filters it out, and if not, it adds it. This approach mirrors common patterns used in React Native bookmarking tutorials (React Native Expert, 2024).

Figure 4:

```
export async function toggleFavourite(product: Product): Promise<void> {
  try {
    const currentFavourites = await getFavourites();
    const existingIndex = currentFavourites.findIndex(
      (fav) => fav.id === product.id,
    );
    let updatedFavourites: FavouriteItem[];
    if (existingIndex !== -1) {
      updatedFavourites = currentFavourites.filter((f) => f.id !== product.id);
    } else {
      updatedFavourites = [
        ...currentFavourites,
        { id: product.id, product, savedAt: Date.now() },
      ];
    }
    await AsyncStorage.setItem(
      FAVOURITES_KEY,
      JSON.stringify(updatedFavourites),
    );
```

- **History Store (historyStore.ts)**

The **historyStore.ts** module handles the HistoryItem array. When a new product is scanned, the **addToHistory()** function searches for an existing item with the same ID and removes it before pushing the new scan to the top of the array. This ensures the history list remains chronological and free of side-by-side duplicates.

Figure 5:

```
export async function addToHistory(
  product: Product,
  alternative?: Product,
): Promise<void> {
  try {
    const currentHistory = await getHistory();
    const filteredHistory = currentHistory.filter((h) => h.id !== product.id);
    const newItem: HistoryItem = {
      id: product.id,
      product,
      scannedAt: Date.now(),
      score: product.score,
      alternative,
    };
    const updatedHistory = [newItem, ...filteredHistory];
    await AsyncStorage.setItem(HISTORY_KEY, JSON.stringify(updatedHistory));
  } catch {}
```

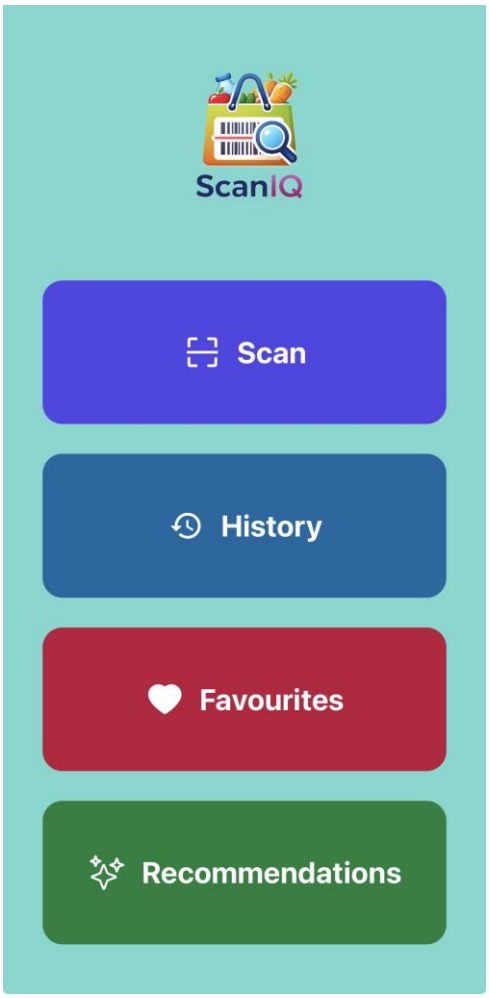
This approach is similar to common patterns used in React Native applications for managing shared data (React Native Expert, 2024).

6.4 Dynamic Routing and UI Layout

Navigation between screens was implemented using **Expo Router**, which uses a file-based routing system. I designed the app using a card-based UI with a shared **BottomNav** component.

The application includes main screens:

- **Scan screen**
- **History screen**
- **Favourites screen**
- **Product details screen**



When a user taps a scanned product or an item in their history list, the app calls:

- **router.push("/product/" + item.id)**

This routes the user to **app/product/[id].tsx**, a dynamic route file that renders the **ProductDetailsScreen**. On this screen, the useLocalSearchParams hook extracts the ID from the URL and fetches the complete product profile from the history store.

To make the health impact immediately clear to the user, I created a custom **colourForLevel** helper function. This function maps the product's safety level to specific hexadecimal colours (e.g., #ef4444 for red/bad, #22c55e for green/good). This logic was inspired by how the Nutri-Score traffic-light system visually penalises negative ingredients while rewarding positive ones (Foodwatch, 2026).

7. Improvements and Enhancements

Once the core scanning and scoring functionality was operational, I evaluated the application against existing market solutions like Yuka to identify areas for improvement. I realized that simply showing a numeric score was not enough to help users make healthy choices. I implemented several major enhancements focused on usability, data analysis, and UI consistency.

7.1 Product Recommendation System

The most significant feature addition was the Product Recommendation engine. Initially, if a user scanned a product with a "Bad" or "Poor" score, they were left without alternatives. To solve this, I developed a system that suggests healthier alternatives through the following process:

- Triggers a secondary request to the OpenFoodFacts API targeting the exact same product category.
- Filters the API response to only include products with higher nutritional grades.
- Applies a slice(0,5) function in the JavaScript array logic to restrict the payload to the top 5 alternatives.

Figure 6:

```
    const results = await Promise.all(
      products.slice(0, 5).map((p: any) => fetchProductByBarcode(p.code)),
    );

    const filtered = results.filter(
      (p): p is ProductFromApi =>
        p !== null &&
        p.score >= 50 && // only consider products with a decent score
        p.id !== scannedProduct.score + 10,
    );
    if (filtered.length === 0) return [];

    return filtered.sort((a, b) => b.score - a.score).slice(0, 5);
  } catch {
    return [];
  }
```

- Drastically reduces latency and ensures the UI remains fast and responsive.

Figure 7:

7.2 Ingredient Analysis Engine

To provide clearer explanations to the user, I developed a custom logic module located in **logic/ingredientAnalysis.ts.** Instead of just passing raw ingredient strings to the screen, I created the **analyzeIngredients()** function to parse the data actively:

- **Negative elements**: Scans the string for keywords like sugar, salt, and complex chemical additives.
- **Positive elements**: Identifies beneficial nutrients such as fibre and protein.
- **Structured output**: Returns the categorized data into distinct positive and negative arrays.
- **Visual rendering**: Feeds the arrays to the **ProductDetailsScreen** so the app can render distinct visual sections explaining exactly why a product received its score.

7.3 Bulk Selection and Deletion (History & Favourites)

During usability testing, I realized that managing saved items one by one was tedious. To fix this, I completely refactored the **HistoryScreen.tsx** and **FavouritesScreen.tsx** components to support bulk deletion. I introduced a new isSelecting boolean state and a selectedIds array to manage the flow:

- **Activating selection:** A long press on any history or favourite item activates selection mode.
- **Selecting items:** Tapping an item adds its ID to the selectedIds array and displays a right-aligned square checkbox on the card.
- **Deleting items:** A custom trash icon built with Expo Ionicons appears in the header.
- **Storage updates:** Tapping the trash icon triggers new functions (**deleteHistoryItems** and **deleteFavouriteItems**) that filter the selected IDs out of the local **AsyncStorage** state simultaneously.

7.4 Architectural Refactoring

As the codebase grew, I refactored the entire project into a strict modular architecture. By moving inline styles into dedicated **StyleSheet.create objects**, the React components became much easier to read and maintain. I separated the code into four distinct layers:

- **API:** Handles all external data fetching and JSON parsing.
- **Logic:** Manages the scoring algorithms and ingredient text analysis.
- **UI:** Contains all React Native visual components and screen layouts.
- **Styles**: Stores external stylesheet files like **scanScreenStyles.ts** to keep components clean.

8. Challenges and Problem Solving

Building this mobile application from scratch was a great learning experience, but it came with a few challenges, particularly regarding asynchronous data handling and state management.

8.1 API Latency and Navigation Errors

When I first implemented the recommendation feature, the app attempted to load dozens of alternative products at once. Because the OpenFoodFacts API is massive, this caused the app to freeze. Sometimes it resulted in navigation errors where the user would tap a button but the screen would not transition. I solved this by utilizing **async/await** promises correctly to ensure the UI thread was not blocked, and by implementing the array limit to restrict the payload size.

8.2 State Management Inconsistencies

A major bug I encountered involved **AsyncStorage** keys. After a user deleted an item from their Favourites list using the new bulk-delete feature, the UI would not immediately update to reflect the change. The deleted item still appeared on the screen until the app was completely refreshed. I fixed this issue by applying the following corrections:

- **Array references:** Passed completely new array references ([...newArray]) to the setState function to force React to trigger a re-render.
- **Storage keys:** Standardized all **AsyncStorage** string identifiers so read and write operations aligned perfectly.

8.3 JSX and UI Structure Bugs

During the UI redesign, I faced multiple JSX structure issues, such as missing closing tags and components overlapping the device status bar. I fixed these layout issues using standard React Native solutions:

- **Screen wrapping:** Wrapped main screens in Expo's SafeAreaView to prevent overlap with the notch and status bar.
- **Standardized spacing:** Applied consistent margin and padding variables inside separated style files to achieve a modern layout across all device sizes.

9. Evaluation and Testing

To ensure the app worked properly and met the initial project objectives, I performed manual testing across multiple phases of development.

9.1 Functional and Integration Testing

I tested the core barcode scanner against various real-world household items to verify data accuracy.

- **Valid items:** Verified that the CameraView correctly decoded standard **EAN-13** and **UPC-A** formats.
- **Invalid barcodes:** Deliberately scanned codes not present in the database to verify the app safely caught exceptions.
- **Fallback UI:** Confirmed the app bypassed API mapping and displayed a "Product not found" message instead of crashing.

9.2 UI/UX and Device Testing

Using the Expo Go client, I ran the application dynamically on a physical iPhone over a local Wi-Fi network. This allowed me to test the application's layout on a real screen.

- **Navigation stability:** Verified the BottomNav bar remained securely anchored at the bottom of the screen.
- **Scroll performance:** Ensured the dynamically loaded lists in the **History** and **Favourites** screens scrolled smoothly without clipping.
- **Visual accessibility:** Evaluated the light-theme colour contrast to confirm the dynamically assigned red, orange, and green health dots were clearly visible against the white product cards.

9.3 Logic Verification

To verify the **ingredientAnalysis.ts** script, I logged the API JSON outputs to the console and compared the raw strings against the arrays generated by my app. I confirmed that hidden sugars (like dextrose and syrups) were successfully caught by the negative ingredient filter. These were heavily penalized in the scoring algorithm, fulfilling the project's goal of exposing unhealthy hidden additives.

10. Conclusion and Future Work

Overall, the development of ScanIQ was a success. The project met its primary objective: bridging the gap between complex nutritional labels and everyday consumers by providing a transparent, visual, and easy-to-understand mobile application. By exploring cross-platform technologies like **React Native** and **Expo Router**, combined with the vast open-

source data of the **OpenFoodFacts API**, I built a functional prototype that respects user privacy by avoiding forced authentication.

This project significantly improved my technical proficiency as a software developer. I gained practical experience in mobile component architecture, **REST API integration**, **asynchronous JavaScript**, and local device storage management.

10.1 Future Enhancements

While the current build of **ScanIQ** is fully functional, there are several avenues for future expansion:

- **Cosmetics Integration:** Integrate the OpenBeautyFacts API to expand the app's capability beyond food, allowing users to scan shampoos, lotions, and soaps for harmful chemical additives.
- **Optical Character Recognition (OCR):** Implement an image-scanning feature to extract text directly from a photo of an ingredient list, allowing the app to score products lacking standard barcodes.
- **Cloud Syncing:** Add an optional Firebase cloud-sync feature to allow users to access their History and Favourites across multiple devices.

References:

- **OpenFoodFacts (2026)** API documentation. Available at: Introduction to Open Food Facts API documentation - Product Opener (Open Food Facts Server)
- **Appinventiv (2026)** How to build an app like Yuka. Available at: How to Build an App Like Yuka in 2026
- ***Expo (2026)*** **Camera API documentation**. Available at: Camera - Expo Documentation
- **Foodwatch (2026)** How the Nutri-Score works. Available at: How the Nutri-Score works | Foodwatch EN
- **React Native Expert (2024)** Navigation in React Native. Available at: Explore a step-by-step guide to build a bottom navigation bar in React Native. Boost user engagement with optimized and responsive navigation.
- **StackOverflow (2022)** Expo camera onBarCodeScanned works only once. Available at: javascript - Expo-camera onBarCodeScanned works only once - Stack Overflow
- **Perplexity (2026)** Perplexity AI . Available at: https://www.perplexity.ai

Appendix A – AI Prompt Log

As stated in the Declaration of Authenticity, Perplexity AI was used as an assistive tool during the research, development, and documentation phases of this project. Below is a log of the key prompts and interactions used, demonstrating how the tool was utilized ethically for debugging, research, and English language translation.

Prompt 1 (Writing Assistance & Translation):

"I have my notes for my final project. My English is not my first language, and I need help to write the report in my own words using the past tense. Here are my notes about how I fixed the history storage bug..."

AI Assistance Provided:

The AI helped translate my rough technical notes into academic English, structuring the sentences properly to form the paragraph found in Section 8.2 (State Management Inconsistencies).

Prompt 2 (Structuring Code Explanations):

"How do I explain the History and Favourites bulk deletion clearly based on my code notes?"

AI Assistance Provided:

The AI helped me format my technical logic (using the isSelecting and selectedIds state) into a readable, bulleted list so the examiner could easily follow the user flow in Section 7.3.

Prompt 3 (Formatting Architecture):

"I need to describe my System Architecture. I have an API, a Logic folder, a UI folder, and Styles. Can you help me write this in a professional way?"

AI Assistance Provided:

The AI translated my basic folder structure notes into the professional architectural breakdown seen in Section 7.4, ensuring correct terminology like 'modular architecture' was used.

Prompt 4 (Researching Technologies):

"I am building a barcode scanning app. What is the best library to use with React Native and Expo to scan barcodes quickly?"

AI Assistance Provided:

The AI suggested using Expo's CameraView component over older deprecated libraries. It

provided a summary of how to use it, which I then used to structure my ScanScreen implementation.

Prompt 5 (Debugging Code):

"I am using AsyncStorage to delete an item from my Favourites list, but the UI is not updating immediately on the screen. I have to refresh the app to see it disappear. How do I fix this React state bug?"

AI Assistance Provided:

The AI explained that React Hooks require a completely new array reference to trigger a re-render. It suggested using the spread operator [...newArray] when calling setState, which successfully helped me resolve my UI rendering bug.